FACE RECOGNITION 1

Christine Dewi, Ph.D.
Department of Information Management,
Chaoyang University of Technology Taichung, Taiwan, R.O.C.
Department of Information Technology,
Satya Wacana Christian University Salatiga, Indonesia.

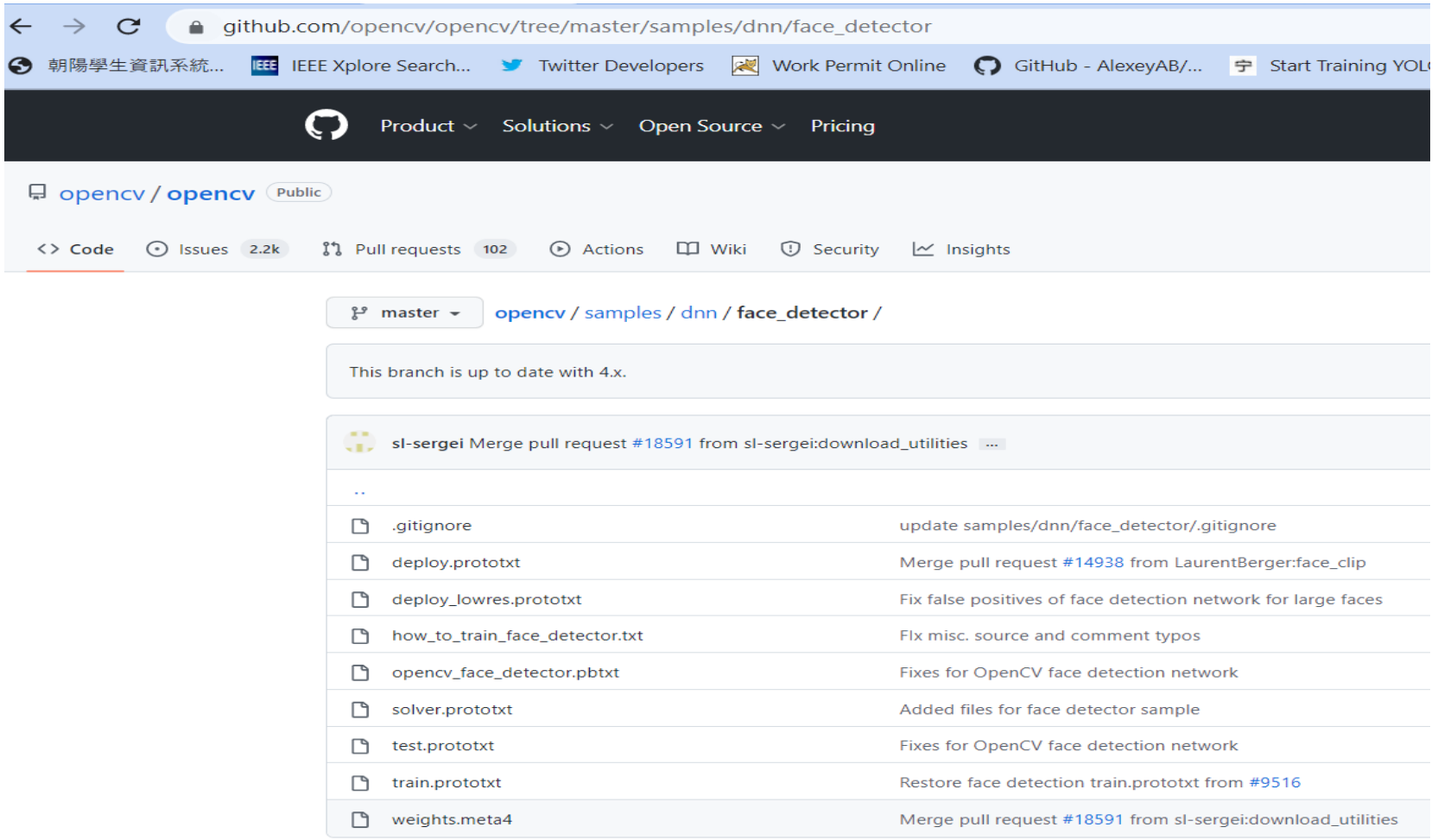
Website : https://sites.google.com/view/christinedewi/home



FACE DETECTION WITH OPENCV AND DEEP LEARNING

- A fast and accurate face detection with OpenCV using a pre-trained deep learning face detector model shipped with the library. Two files are required for using OpenCV's deep neural network module with Caffe models.
- The .prototxt file (define the model architecture i.e., the layers themselves)
- The .caffemodel file (which contains the weights for the actual layers)
- You can find Caffe-based face detector prototxt files in the face_detector sub-directory of the dnn samples.

FACE DETECTION WITH OPENCV AND DEEP LEARNING



Website : https://sites.google.com/view/christinedewi/home

INSTALL

- Python 3.5
- OpenCV
  - conda install -c conda-forge dlib
  - Python 3.5
  - Install opencv
  - conda install tensorflow
  - Conda install h5py
- pip install numpy opencv-python dlib imutils

RUN

- To detect face from photos:
- python detect_faces_image.py images/image1.jpg

dataSet.py | LICENSE | dataSet.py | testimage.py | traintest.py | model.h5 | deploy.prototxt.txt | detect_faces_image.py

```python
# import libraries
import cv2
import sys
import numpy as np

def load_model(image_path):
    # load serialized model from disk
    net = cv2.dnn.readNetFromCaffe('deploy.prototxt.txt','res10_300x300_ssd_iter_140000.caffemodel')

    image = cv2.imread(image_path)
    (h, w) = image.shape[:2]
    blob = cv2.dnn.blobFromImage(cv2.resize(image, (300, 300)), 1.0,
        (300, 300), (104.0, 177.0, 123.0))

    # pass the blob through the network and obtain the detections and
    # predictions
    net.setInput(blob)
    detections = net.forward()

    # loop over the detections
    for i in range(0, detections.shape[2]):
        # extract the confidence (i.e., probability) associated with the
        # prediction
        confidence = detections[0, 0, i, 2]

        # filter out weak detections by ensuring the `confidence` is greater than the minimum confidence
        '''you can also change the 'confidence' (0.5 here) for better results'''
        if confidence > 0.5:
            # compute the (x, y) coordinates of the bounding box for the object
            box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
            (startX, startY, endX, endY) = box.astype("int")
            #print('detections', detections[0, 0, i, 3:7])
```

OUTPUT





Website : https://sites.google.com/view/christinedewi/home

TRY

- To detect face from videos:
- python detect_faces_video.py

dataSet.py | LICENSE | dataSet.py | testimage.py | traintest.py | model.h5 | deploy.prototxt.txt | detect_faces_image.py | detect_faces_video.py

```
# import libraries
import cv2
import time
import imutils
import numpy as np
from imutils.video import VideoStream


# load serialized model from disk and initialize the video stream
net = cv2.dnn.readNetFromCaffe('deploy.prototxt.txt','res10_300x300_ssd_iter_140000.caffemodel')
vs = VideoStream(src=0).start()
time.sleep(2.0)

# loop over the frames from the video stream
while True:
    # grab the frame from the threaded video stream and resize it to have a maximum width of 400 pixels
    frame = vs.read()
    frame = imutils.resize(frame, width=400)

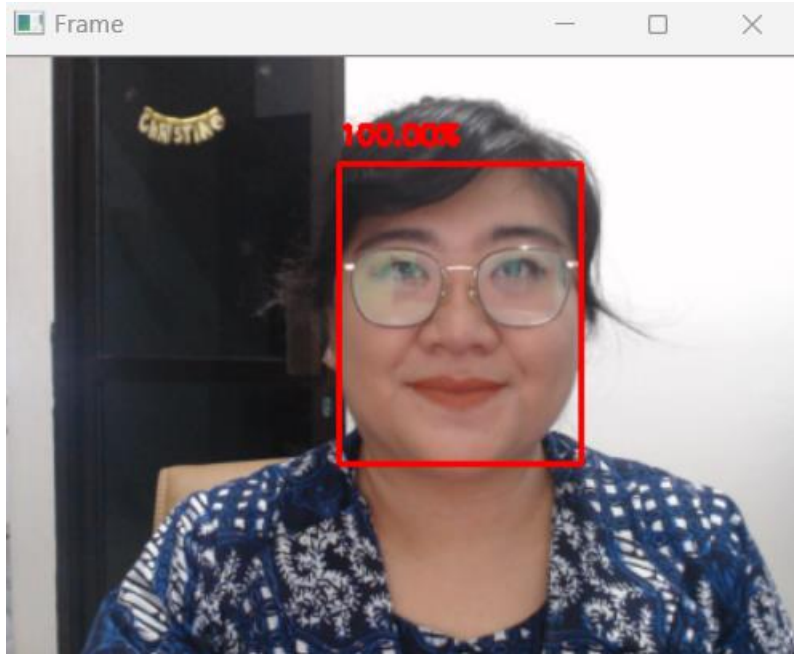
    # grab the frame dimensions and convert it to a blob
    (h, w) = frame.shape[:2]
    blob = cv2.dnn.blobFromImage(cv2.resize(frame, (300, 300)), 1.0, (300, 300), (104.0, 177.0, 123.0))

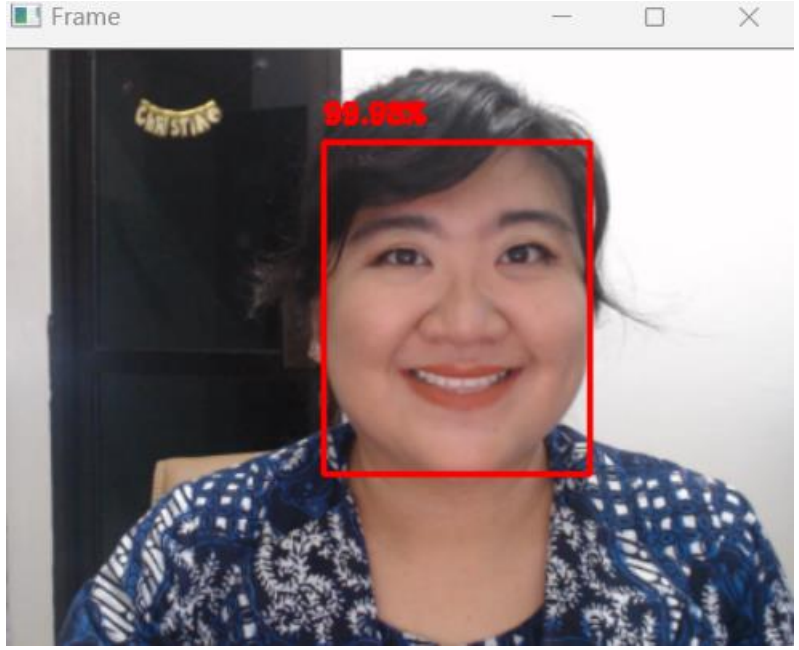
    # pass the blob through the network and obtain the detections and predictions
    net.setInput(blob)
    detections = net.forward()

    # loop over the detections
    for i in range(0, detections.shape[2]):
        # extract the confidence (i.e., probability) associated with the prediction
        confidence = detections[0, 0, i, 2]
```

OUTPUT

- To detect face from videos:
- python detect_faces_video.py





TRY

- python main.py

dataSet.py | LICENSE | dataSet.py | testimage.py | traintest.py | model.h5 | deploy.prototxt.txt | detect_faces_image.py | detect_faces_video.py | main.py

```python
# import the necessary packages
from imutils import face_utils
import dlib
import cv2

# initialize dlib's face detector (HOG-based) and then create
# the facial landmark predictor
p = "shape_predictor_68_face_landmarks.dat"
detector = dlib.get_frontal_face_detector()
predictor = dlib.shape_predictor(p)

cap = cv2.VideoCapture(0)

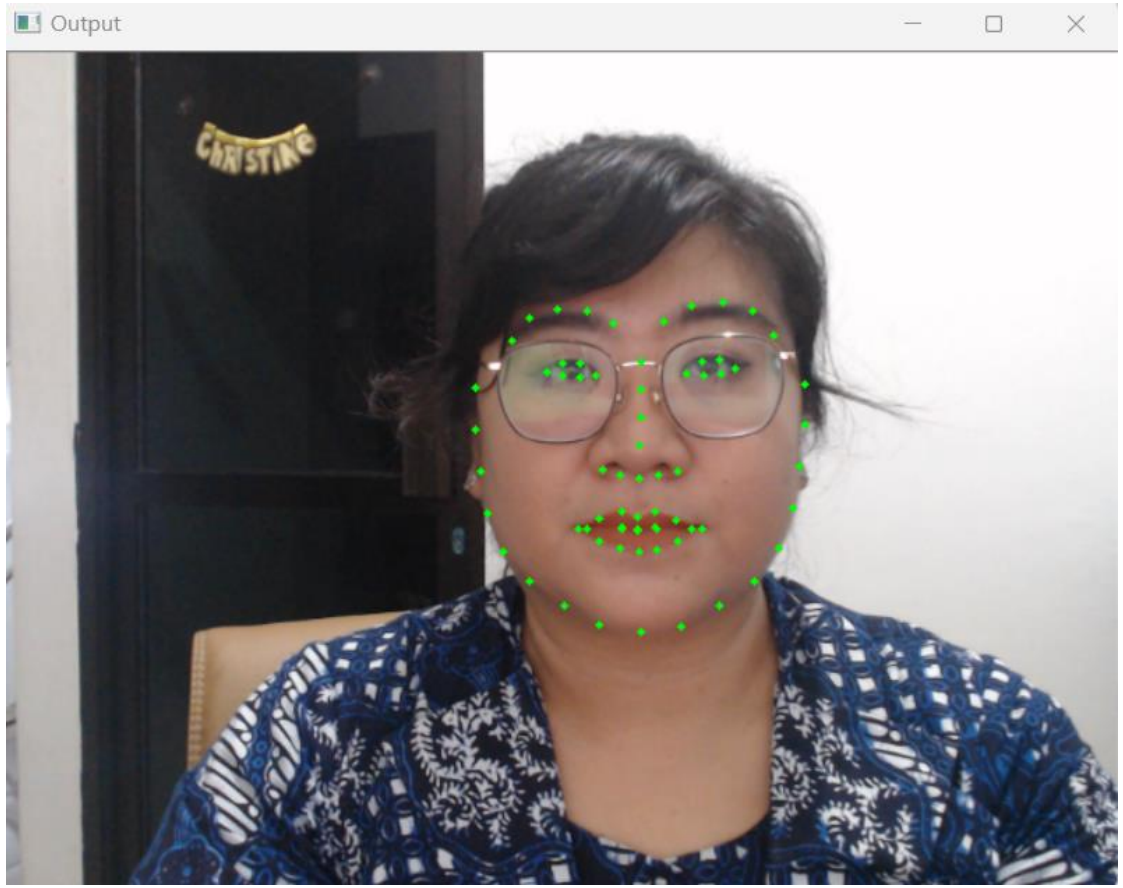
while True:
    # load the input image and convert it to grayscale
    _, image = cap.read()
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # detect faces in the grayscale image
    rects = detector(gray, 0)

    # loop over the face detections
    for (i, rect) in enumerate(rects):
        # determine the facial landmarks for the face region, then
        # convert the facial landmark (x, y)-coordinates to a NumPy
        # array
        shape = predictor(gray, rect)
        shape = face_utils.shape_to_np(shape)
```

OUTPUT

- python main.py



Website : https://sites.google.com/view/christinedewi/home

OUTPUT:



Input

Output

Website : https://sites.google.com/view/christinedewi/home

FACE RECOGNITION 2

Christine Dewi, Ph.D.
Department of Information Management,
Chaoyang University of Technology Taichung, Taiwan, R.O.C.
Department of Information Technology,
Satya Wacana Christian University Salatiga, Indonesia.

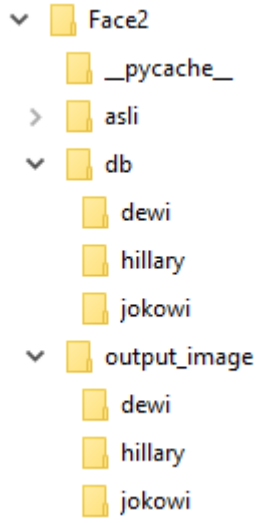
Website :
https://sites.google.com/view/christinedewi/home



PREPARE THE IMAGE 1

- db/dewi



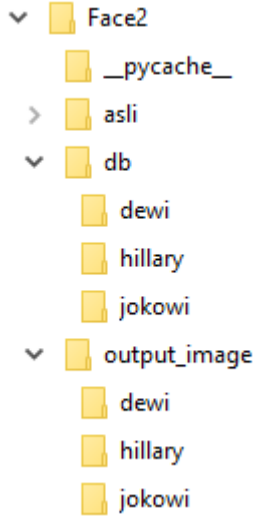


Website : https://sites.google.com/view/christinedewi/home

PREPARE THE IMAGE 2

- db/hillary

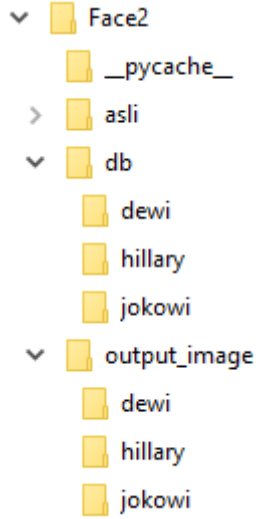




PREPARE THE IMAGE 3

- db/jokowi





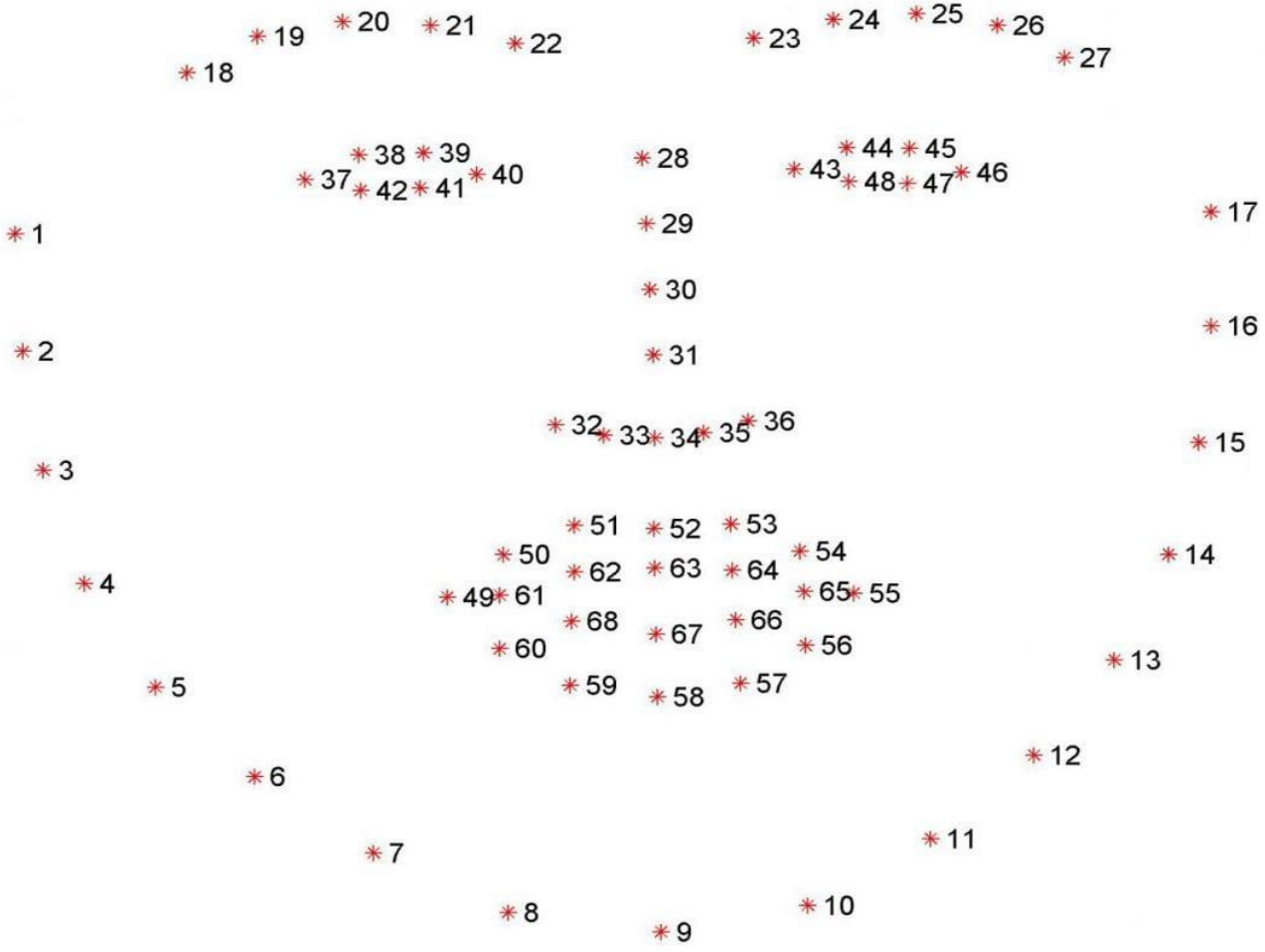
Website : https://sites.google.com/view/christinedewi/home

INSTALL LIBRARY

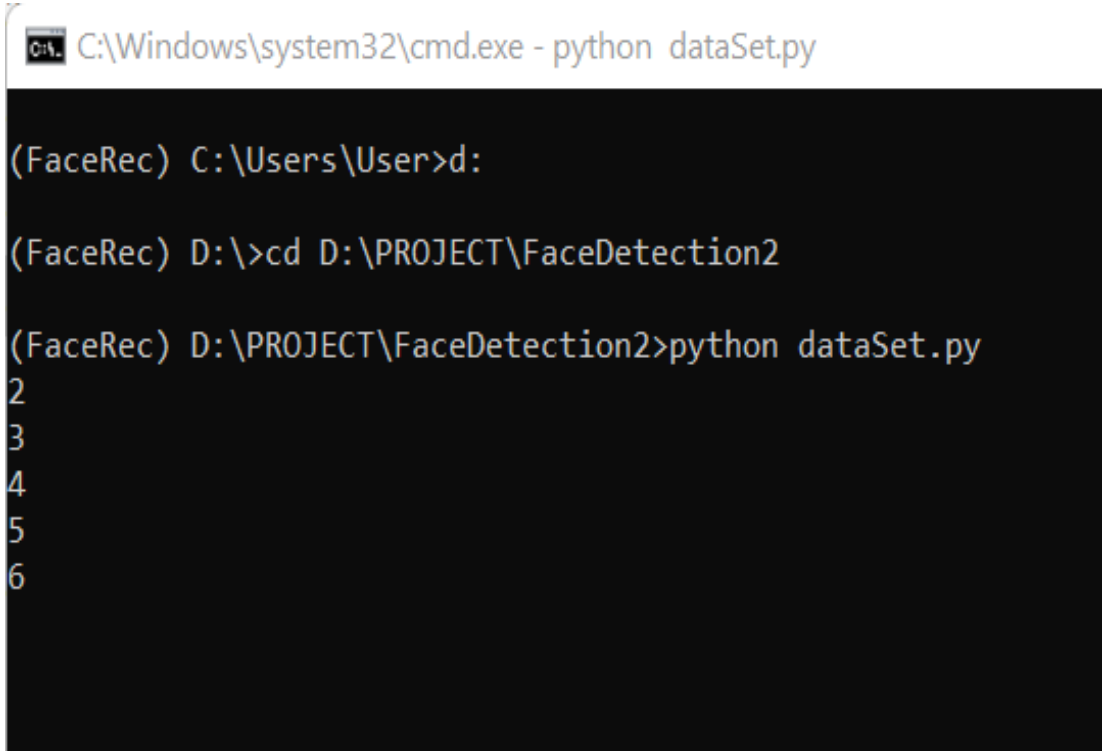
- conda install -c conda-forge dlib
- Python 3.5
- Install opencv
- conda install tensorflow
- Conda install h5py

SHAPE_PREDICTOR_68_FACE_LANDMARKS.DAT

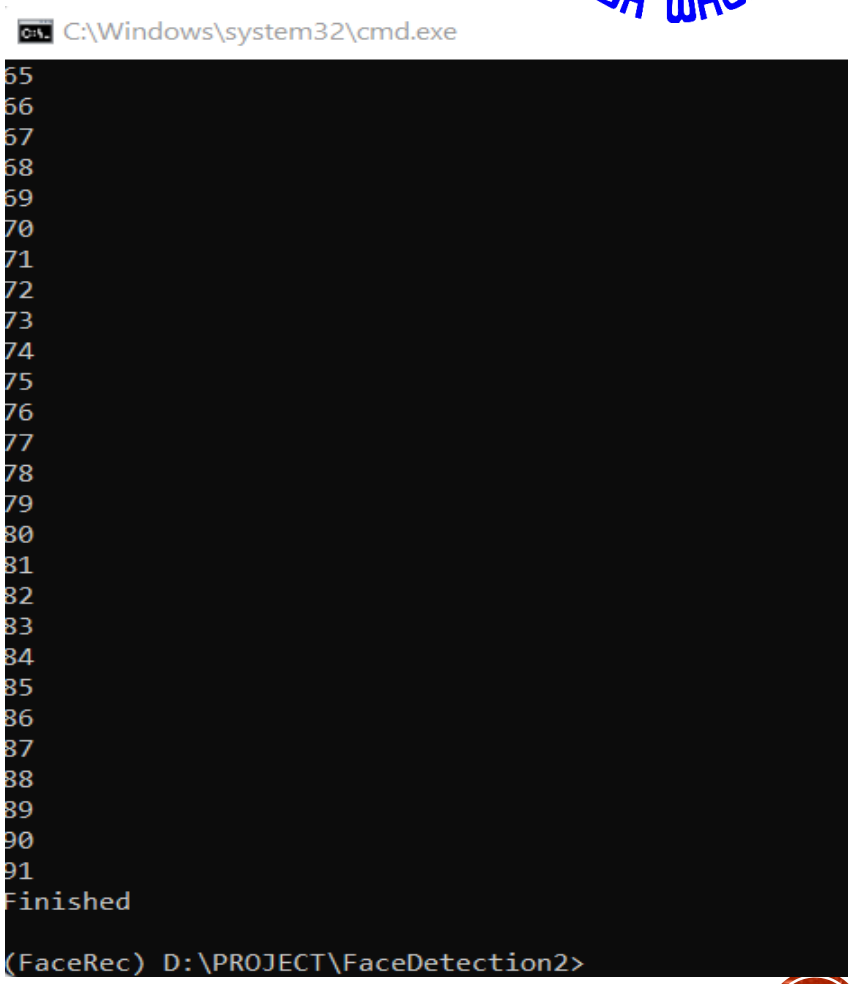
The pre-trained facial landmark detector inside the dlib library is used to estimate the location of ***68 (x, y)-coordinates*** that map to facial structures on the face.



RUN



python dataSet.py



PREPARE THE OUTPUT IMAGE USING DLIB, OPENCV

```python
import cv2
import dlib
import os
import sys
import random

#ouput_image folder
output_dir = 'output_image'
size = 64

if not os.path.exists(output_dir):
    os.makedirs(output_dir)
# 改变图片的亮度与对比度
def relight(img, light=1, bias=0):
    w = img.shape[1]
    h = img.shape[0]

    for i in range(0,w):
        for j in range(0,h):
            for c in range(3):
                tmp = int(img[j,i,c]*light + bias)
                if tmp > 255:
                    tmp = 255
                elif tmp < 0:
                    tmp = 0
                img[j,i,c] = tmp
    return img

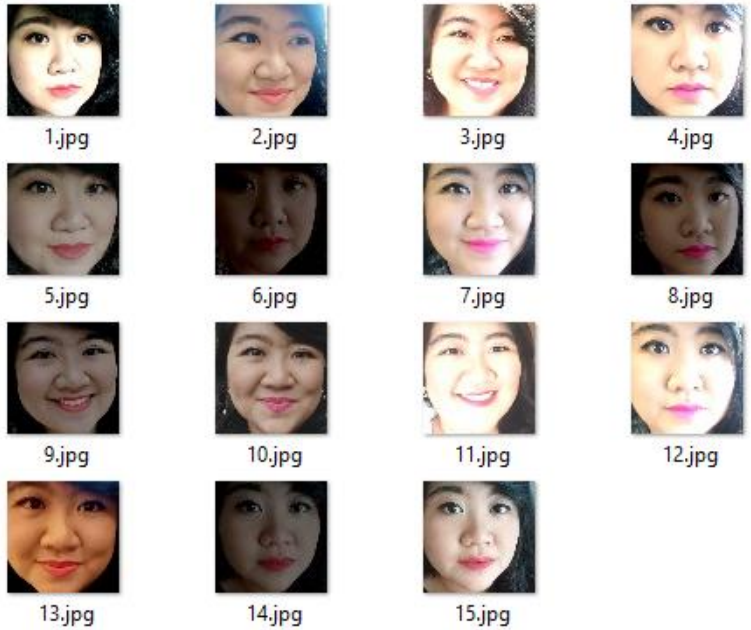
```

(dataSet.py)

```python
#使用dlib自带的frontal_face_detector作为我们的特征提取器 dlib for face detector
detector = dlib.get_frontal_face_detector()
predictor = dlib.shape_predictor( 'shape_predictor_68_face_landmarks.dat')
index = 1
people = [people for people in os.listdir("db/")]
for people in people:
    images = [image for image in os.listdir("db/" + people + '/')]
    for image in images:
        # 转为灰度图片
        img = cv2.imread("db/" + people+ '/' +image)
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        # 使用detector进行人脸检测
        dets = detector(img, 0)
        for i, d in enumerate(dets):
            x1 = d.top() if d.top() > 0 else 0
            y1 = d.bottom() if d.bottom() > 0 else 0
            x2 = d.left() if d.left() > 0 else 0
            y2 = d.right() if d.right() > 0 else 0
            face = img[x1:y1,x2:y2]
            # 调整图片的对比度与亮度， 对比度与亮度值都取随机数，这样能增加样本的多样性
            face = relight(face, random.uniform(0.5, 1.5), random.randint(-50, 50))
            face = cv2.resize(face, (size,size))
            cv2.imwrite(output_dir+'/'+ people +'/'+str(index)+'.jpg', face)
        index += 1
        print(index)
    print('Finished')
```

OUTPUT_IMAGE

1



2



3



Website : https://sites.google.com/view/christinedewi/home

TRAIN THE WEIGHT & MODEL FROM OUTPUT_IMAGE) = USING TENSORFLOW, KERAS

```
"""
Created on Fri Apr 19 15:39:21 2019
@author: Christine
"""
import cv2
import os
import numpy as np

from tensorflow.python.keras.utils import np_utils
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.python.keras.constraints import maxnorm
from tensorflow.keras.optimizers import SGD
from tensorflow.python.keras.layers.convolutional import Conv2D
from tensorflow.python.keras.layers.convolutional import MaxPooling2D

# Saving and loading model and weights
from tensorflow.python.keras.models import model_from_json
from tensorflow.python.keras.models import load_model
from sklearn.utils import shuffle
from sklearn.model_selection import train_test_split
# load and prepare data
people = [people for people in os.listdir("output_image/")]
print(people)
num_classes = 3
img_data_list = []
labels = []
valid_images = [".jpg",".gif",".png"]
```

python traintest.py

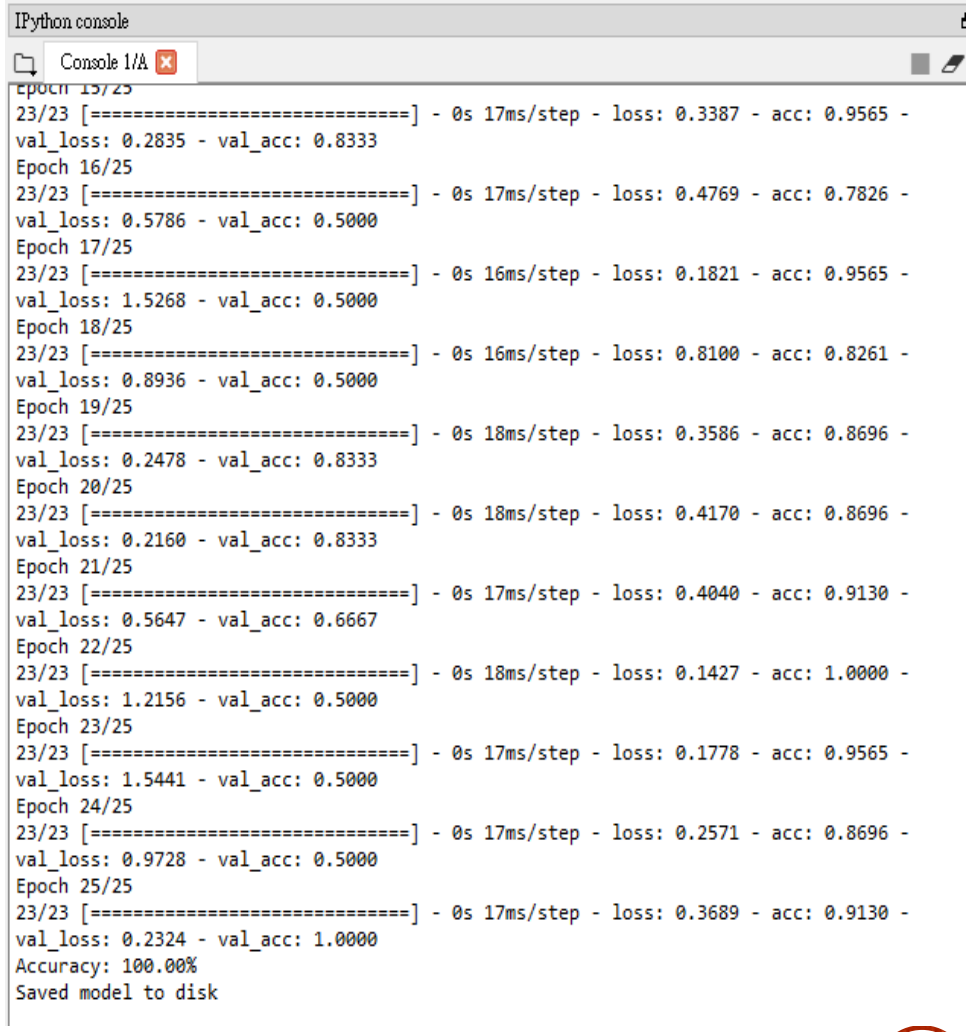
```
for index, person in enumerate(people):
  print(index)
  dir_path = 'output_image/' + person
  print(dir_path)
  for img_path in os.listdir(dir_path):
    name, ext = os.path.splitext(img_path)
    if ext.lower() not in valid_images:
        continue
    img_data = cv2.imread(dir_path + '/' + img_path)
    # convert image to gray
    img_data=cv2.cvtColor(img_data, cv2.COLOR_BGR2GRAY)
    img_data_list.append(img_data)
    labels.append(index)
img_data = np.array(img_data_list)
img_data = img_data.astype('float32')

labels = np.array(labels ,dtype='int64')
# scale down(so easy to work with)
img_data /= 255.0
img_data= np.expand_dims(img_data, axis=3)
# convert class labels to on-hot encoding
Y = np_utils.to_categorical(labels, num_classes)
#Shuffle the dataset
x,y = shuffle(img_data,Y, random_state=2)
# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=2)
# Defining the model
input_shape=img_data[0].shape
print(input_shape)
model = Sequential()
model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=input_shape, padding='same', activation='relu',
                 kernel_constraint=maxnorm(3)))
model.add(Dropout(0.2))
model.add(Conv2D(32, (3, 3), activation='relu', padding='same', kernel_constraint=maxnorm(3)))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(512, activation='relu', kernel_constraint=maxnorm(3)))
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))
```

TRAIN THE WEIGHT & MODEL FROM OUTPUT_IMAGE)

```
73 # Compile model
74 epochs = 25
75 lrate = 0.01
76 decay = lrate/epochs
77 sgd = SGD(lr=lrate, momentum=0.9, decay=decay, nesterov=False)
78 model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])
79 print(model.summary())
80
81 # Fit the model
82 model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=epochs, batch_size=32)
83 # Final evaluation of the model
84 scores = model.evaluate(X_test, y_test, verbose=0)
85 print("Accuracy: %.2f%%" % (scores[1]*100))
86 # serialize model to JSON
87 model_json = model.to_json()
88 with open("model.json", "w") as json_file:
89     json_file.write(model_json)
90 # serialize weights to HDF5
91 model.save_weights("model.h5")
92 print("Saved model to disk")
```

python traintest.py



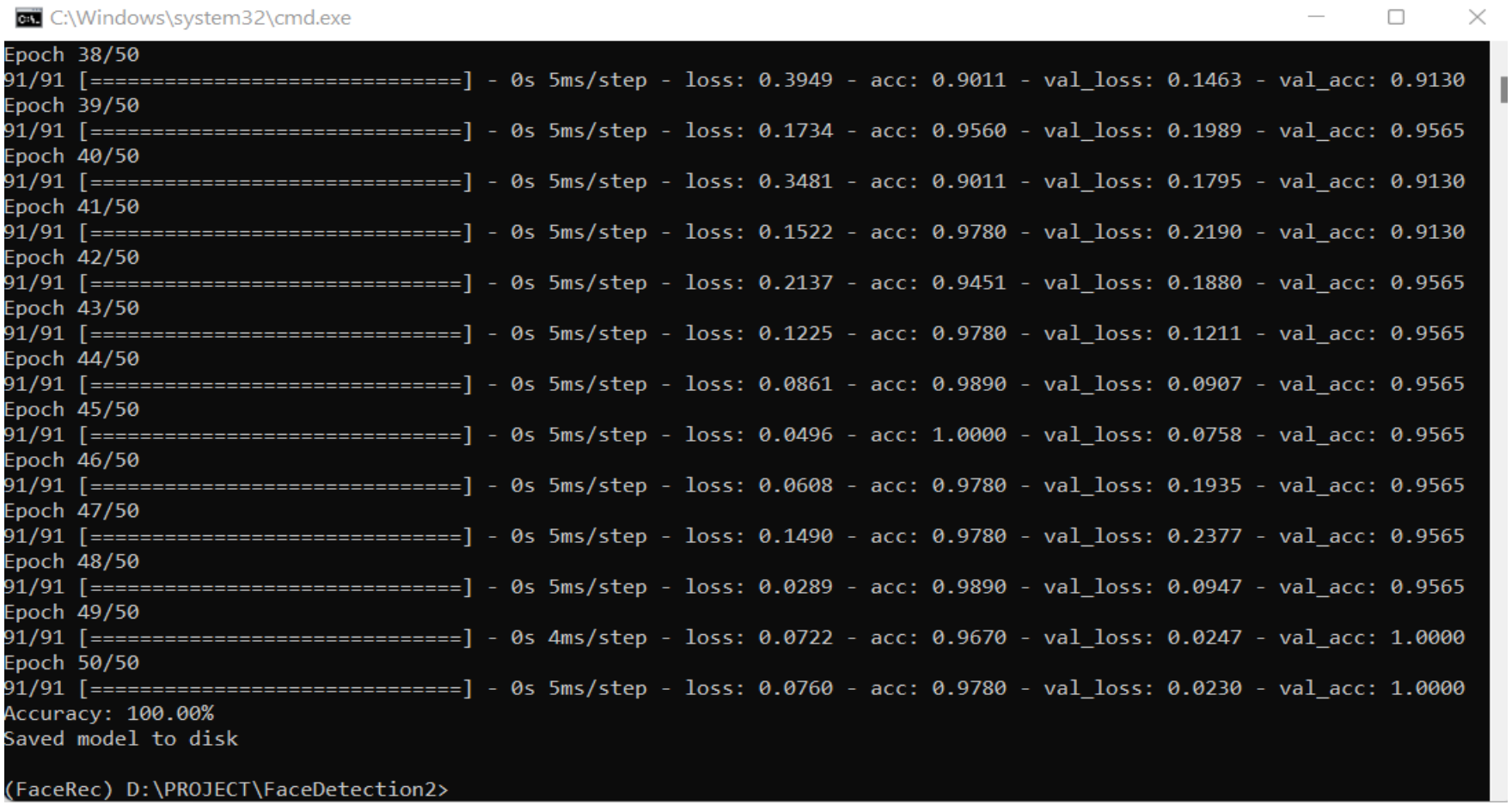
TRAIN THE WEIGHT & MODEL FROM OUTPUT_IMAGE)

```
0
output_image/dewi
1
output_image/hillary
2
output_image/jokowi
(64, 64, 1)
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 64, 64, 32)        320
_________________________________________________________________
dropout (Dropout)            (None, 64, 64, 32)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 64, 64, 32)        9248
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 32, 32, 32)        0
_________________________________________________________________
flatten (Flatten)            (None, 32768)             0
_________________________________________________________________
dense (Dense)                (None, 512)               16777728
_________________________________________________________________
dropout_1 (Dropout)          (None, 512)               0
_________________________________________________________________
dense_1 (Dense)              (None, 3)                 1539
=================================================================
Total params: 16,788,835
Trainable params: 16,788,835
Non-trainable params: 0
```

python traintest.py

TRAIN THE WEIGHT & MODEL FROM OUTPUT_IMAGE)

python traintest.py



PREPARE FOR TESTING

```
"""
Created on Fri Apr 19 17:27:50 2019
@author: Christine
"""
from imutils import face_utils
from imutils.face_utils import FaceAligner
import imutils
import time
import cv2
import dlib
import os
import numpy as np

# Saving and loading model and weights
from tensorflow.python.keras.models import model_from_json
from tensorflow.python.keras.models import load_model

# load json and create model
json_file = open('model.json', 'r')
loaded_model_json = json_file.read()
json_file.close()
loaded_model = model_from_json(loaded_model_json)
# load weights into new model
loaded_model.load_weights("model.h5")
print("Loaded model from disk")

print("[INFO] loading facial landmark predictor...")
detector = dlib.get_frontal_face_detector()
predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")
fa = FaceAligner(predictor, desiredFaceWidth=64)
```

python testimage.py

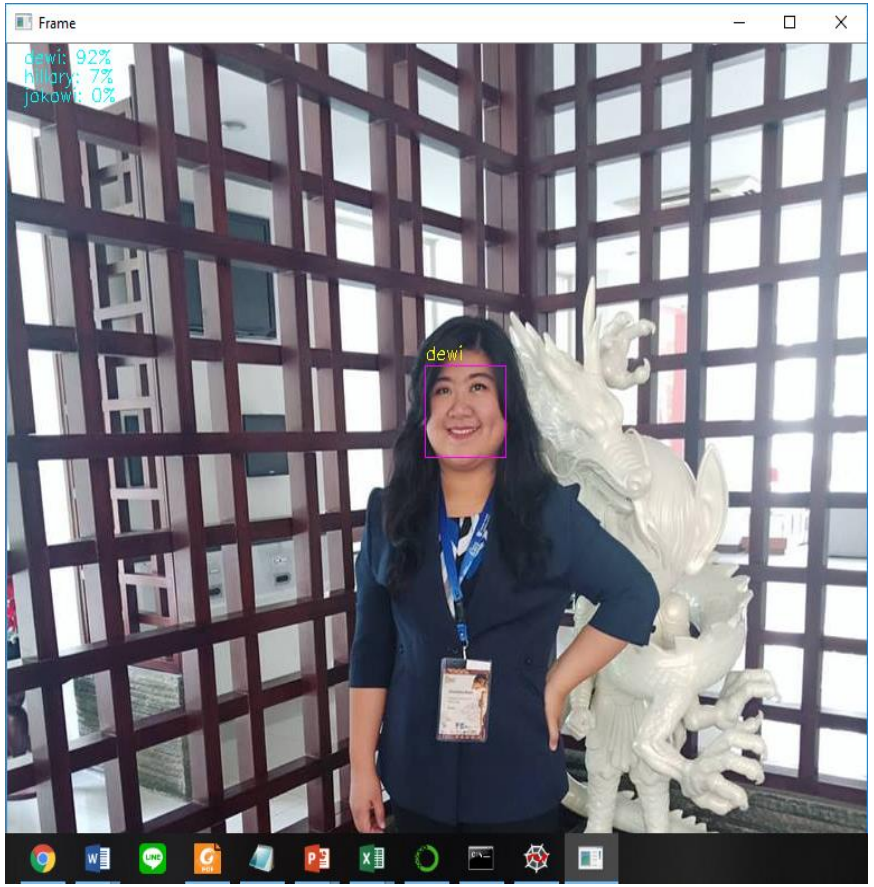
```
people = [people for people in os.listdir("output_image/")]
# loop over the frames from the video stream
text_list = ['image1.jpg', 'image2.jpg', 'image3.jpg', 'image4.jpg', 'image5.jpg', 'image6.jpg']
for i in text_list:
    while True:
        key = cv2.waitKey(1) & 0xFF
         # if the `e` key was pressed, break from the loop
        if key == ord("e"):
            break
        #frame = vs.read()
        #frame = imutils.resize(frame, width=800)
        #height, width = frame.shape[:2]
        img_data = cv2.imread(i)
        gray_frame = cv2.cvtColor(img_data, cv2.COLOR_BGR2GRAY)
         # detect faces in the grayayscale frame
        rects = detector(gray_frame, 0)

        # loopop over the face detections
        for rect in rects:
            faceAligned = fa.align(img_data, gray_frame, rect)
            faceAligned = cv2.cvtColor(faceAligned, cv2.COLOR_BGR2GRAY)
            faceAligned = np.array(faceAligned)
            faceAligned = faceAligned.astype('float32')
            faceAligned /= 255.0
            faceAligned= np.expand_dims([faceAligned], axis=3)

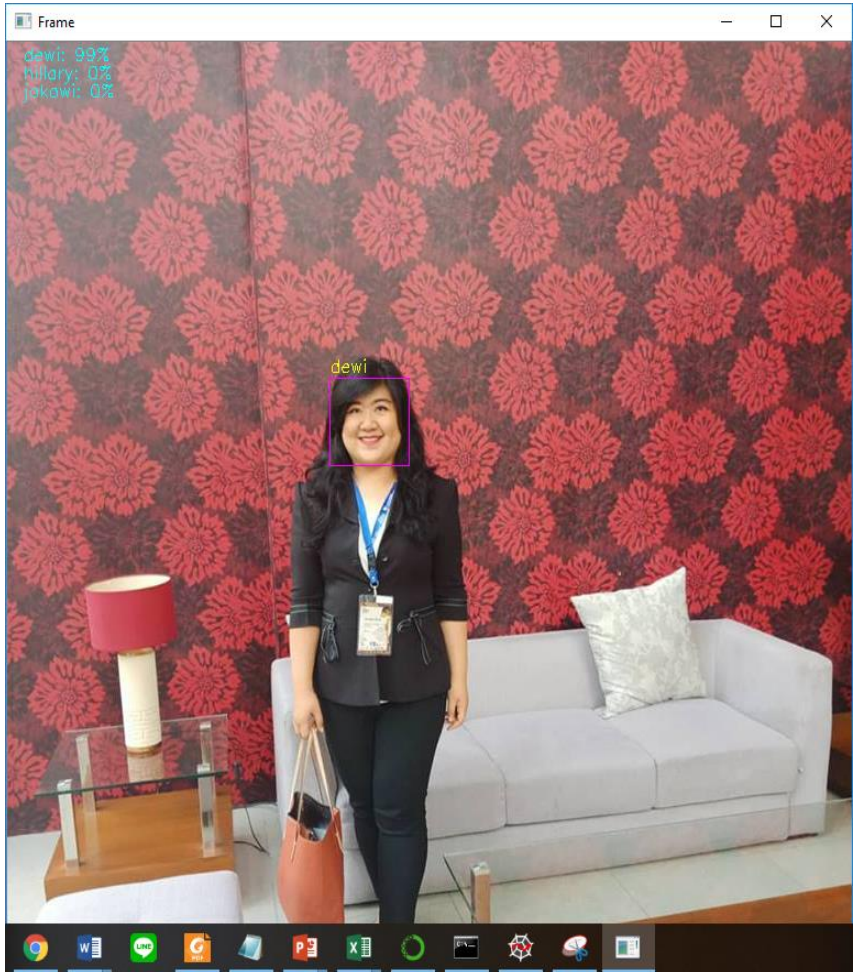
            Y_pred =  loaded_model.predict(faceAligned)
            for index, value in enumerate(Y_pred[0]):
              result = people[index] + ': ' + str(int(value * 100)) + '%'
              cv2.putText(img_data, result, (10, 15 * (index + 1)), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                          (0,0,0), 1)
            # draw rect around face
            (x,y,w,h) = face_utils.rect_to_bb(rect)
            cv2.rectangle(img_data, (x, y), (x+w, y+h), (0,0,255), 1)
            # draw person name
            result = np.argmax(Y_pred, axis=1)
            cv2.putText(img_data, people[result[0]], (x, y - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                        (0,255,0), 1)
        # show the frame
        cv2.imshow("Frame", img_data)

# do a bit of cleanup
cv2.destroyAllWindows()
#vs.stop()
```

RESULT DEWI



Dewi : 92%
Hillary : 7%
Jokowi : 0%



Dewi : 99%
Hillary : 0%
Jokowi : 0%

Website : https://sites.google.com/view/christinedewi/home

RESULT JOKOWI



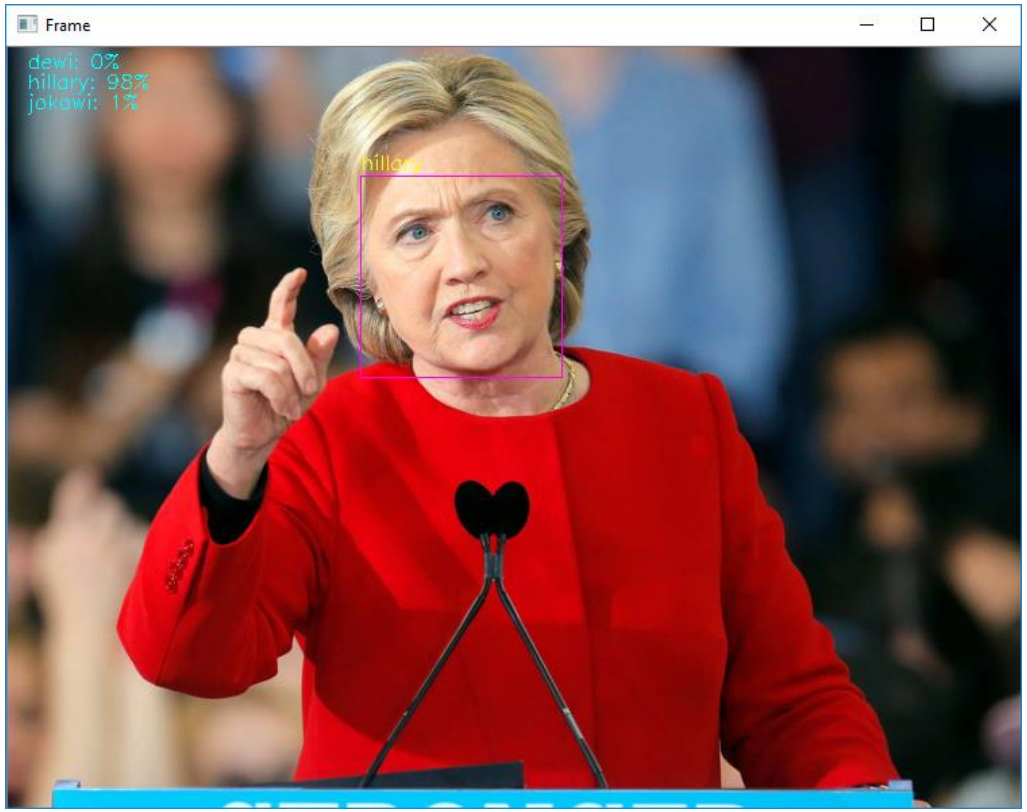
Dewi : 0%
Hillary : 0%
Jokowi : 99%



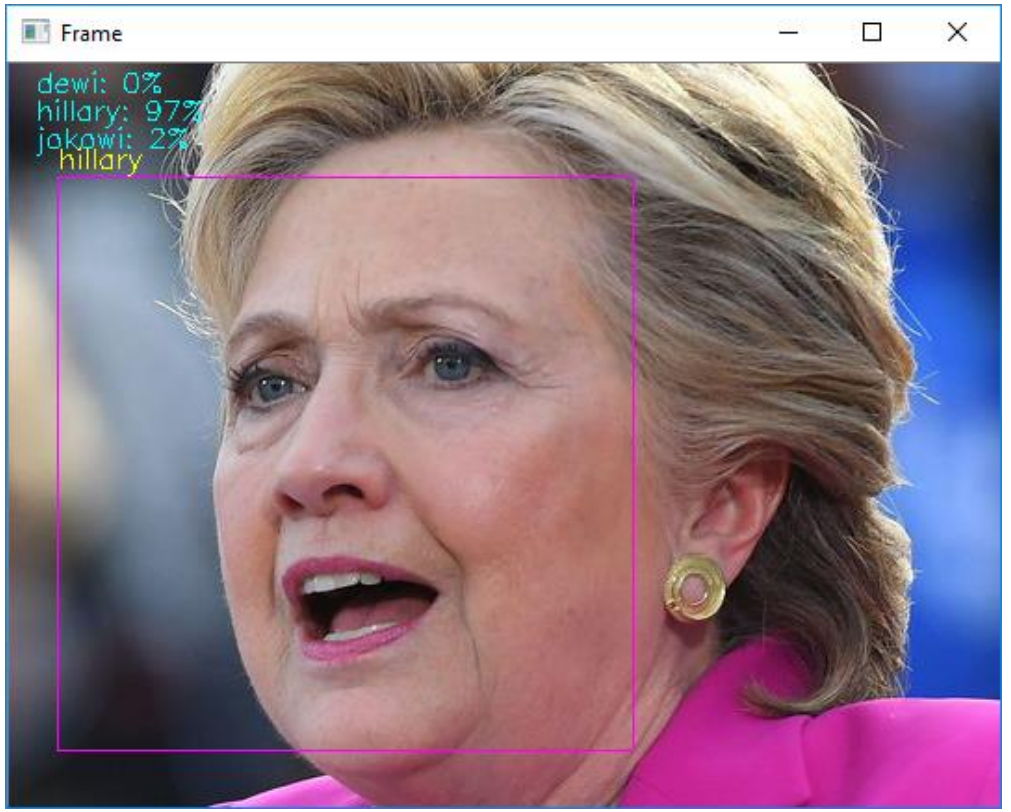
Dewi : 0%
Hillary : 0%
Jokowi : 99%

Website : https://sites.google.com/view/christinedewi/home

RESULT HILLARY



Dewi : 0%
Hillary : 98%
Jokowi : 1%



Dewi : 0%
Hillary : 97%
Jokowi : 2%

Website : https://sites.google.com/view/christinedewi/home

FACE RECOGNITION 3

Face Recognition using OpenCV

Christine Dewi, Ph.D.
Department of Information Management,
Chaoyang University of Technology Taichung, Taiwan, R.O.C.
Department of Information Technology,
Satya Wacana Christian University Salatiga, Indonesia.

Website : https://sites.google.com/view/christinedewi/home



FACE RECOGNITION USING OPENCV

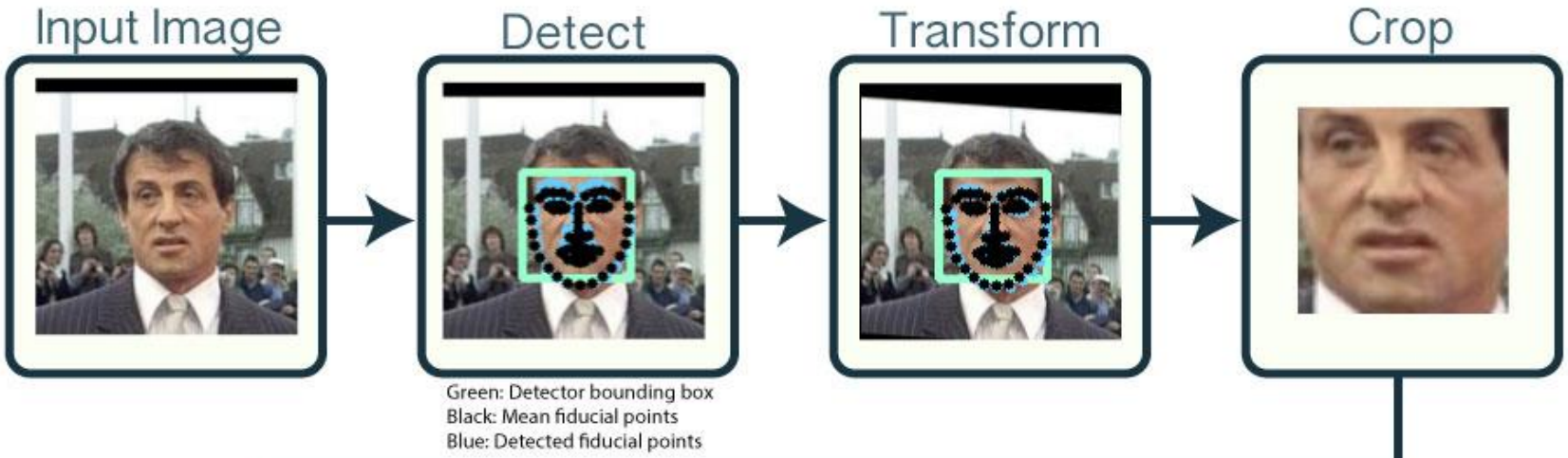


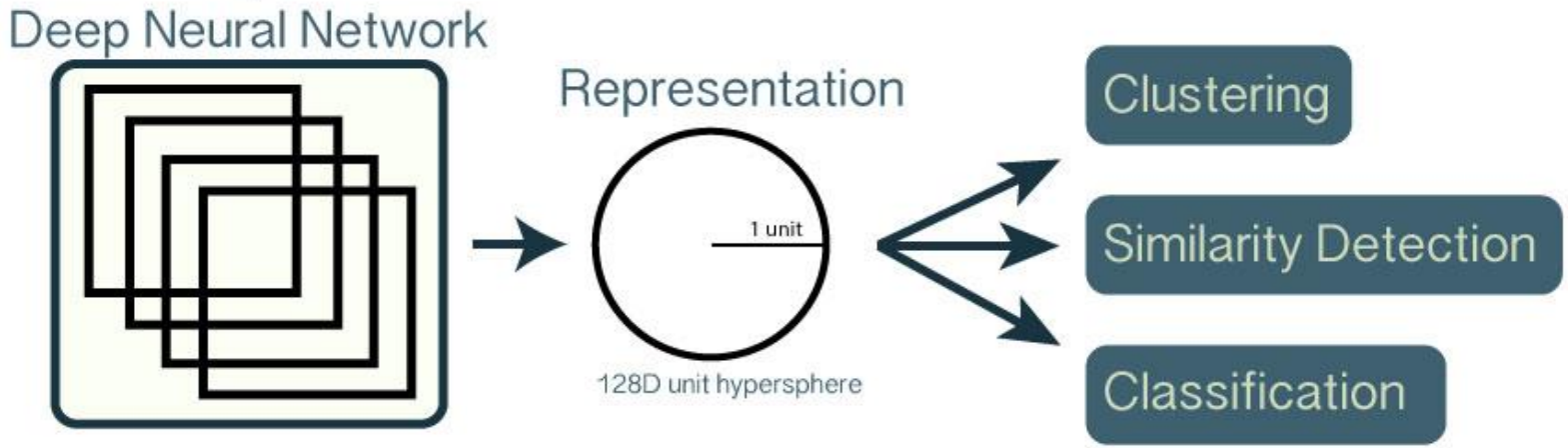
1. Create dataset of face images
2. Detect faces
   using deploy.prototxt and res10_300x300_ssd_iter_140000.caff emodel. (Learn more about face detection)
3. Extract face embeddings for each face present in the image
   using pretrained OpenFace model openface_nn4.small2.v1.t7.
4. Train a SVM model on the face embeddings to recognize faces

OVERVIEW OF OPENFACE FOR A SINGLE INPUT IMAGE

1. Detect faces with a pre-trained models from dlib or OpenCV.
2. Transform the face for the neural network. This repository uses dlib's real-time pose estimation with OpenCV's affine transformation to try to make the eyes and bottom lip appear in the same location on each image.
3. Use a deep neural network to represent (or embed) the face on a 128-dimensional unit hypersphere. The embedding is a generic representation for anybody's face.
4. Unlike other face representations, this embedding has the nice property that a larger distance between two face embeddings means that the faces are likely not of the same person. This property makes clustering, similarity detection, and classification tasks easier than other face recognition techniques where the Euclidean distance between features is not meaningful.
5. Apply clustering or classification techniques to the features to complete the face recognition task.

OVERVIEW OF OPEN FACE FOR A SINGLE INPUT IMAGE



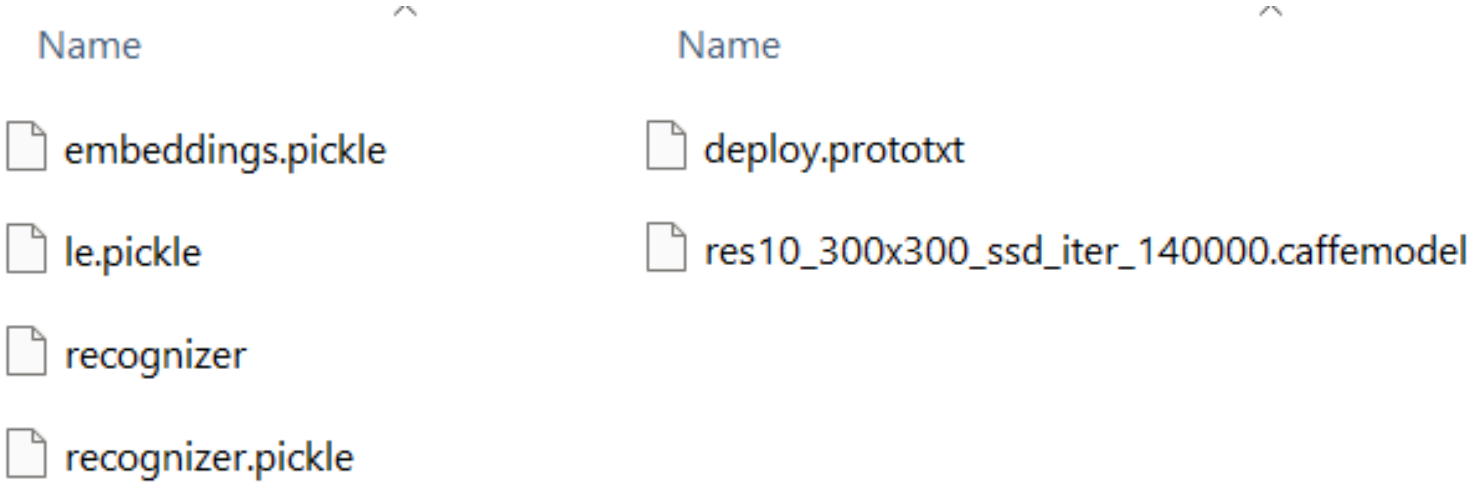


Website : https://sites.google.com/view/christinedewi/home

https://cmusatyalab.github.io/openface/

EXPERIMENT

1. Create dataset of face images.
2. Place the face images in dataset folder.
3. Extract facial embeddings. python extract_embeddings.py
4. Train the SVM model python train_model.py
5. Test the model python recognize_video.py



EXPERIMENT

Extract facial embeddings. python extract_embeddings.py

extract_embeddings.py

```python
# import libraries
from imutils import paths
import numpy as np
import argparse
import imutils
import pickle
import cv2
import os

# load serialized face detector
print("Loading Face Detector...")
protoPath = "face_detection_model/deploy.prototxt"
modelPath = "face_detection_model/res10_300x300_ssd_iter_140000.caffemodel"
detector = cv2.dnn.readNetFromCaffe(protoPath, modelPath)

# load serialized face embedding model
print("Loading Face Recognizer...")
embedder = cv2.dnn.readNetFromTorch("openface_nn4.small2.v1.t7")

# grab the paths to the input images in our dataset
print("Quantifying Faces...")
imagePaths = list(paths.list_images("dataset"))

# initialize our lists of extracted facial embeddings and corresponding people names
knownEmbeddings = []
knownNames = []

# initialize the total number of faces processed
```

EXPERIMENT

Train the SVM model python
train_model.py

extract_embeddings.py | train_model.py

```python
# USAGE
# python train_model.py --embeddings output/embeddings.pickle --recognizer

# import the necessary packages
from sklearn.preprocessing import LabelEncoder
from sklearn.svm import SVC
import argparse
import pickle

# load the face embeddings
print("[INFO] loading face embeddings...")
data = pickle.loads(open("output/embeddings.pickle", "rb").read())

# encode the labels
print("[INFO] encoding labels...")
le = LabelEncoder()
labels = le.fit_transform(data["names"])

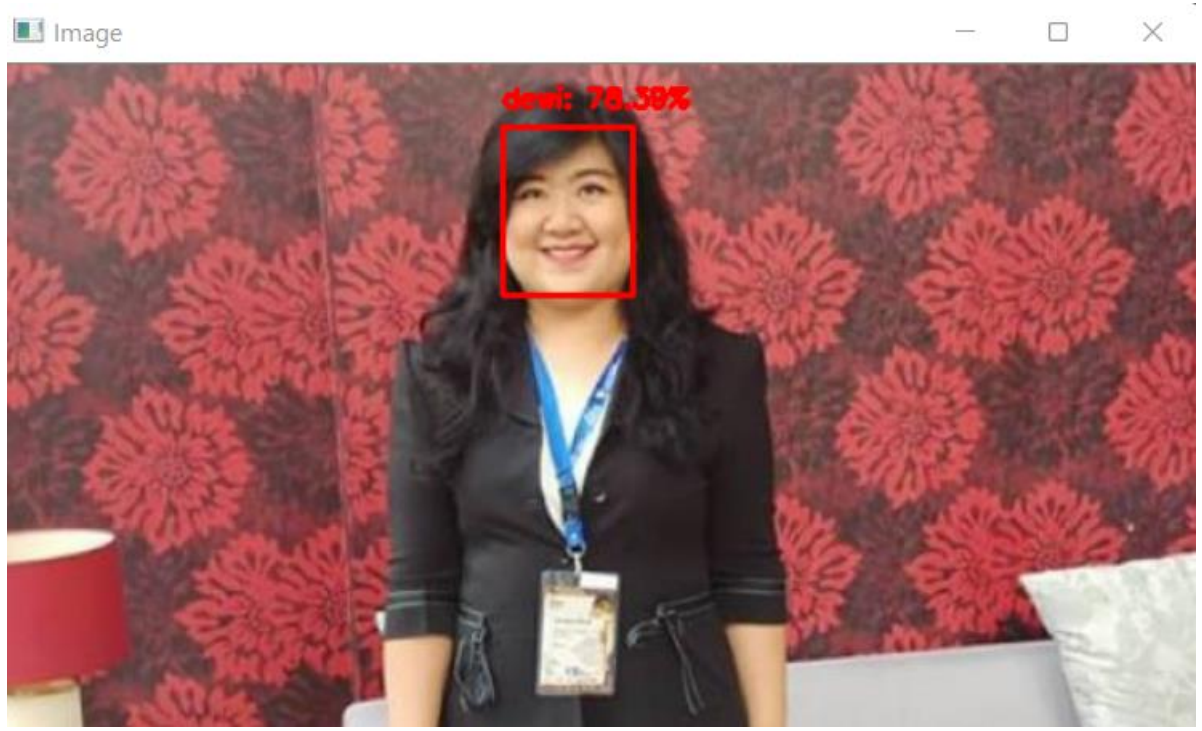
# train the model used to accept the 128-d embeddings of the face and
# then produce the actual face recognition
print("[INFO] training model...")
recognizer = SVC(C=1.0, kernel="linear", probability=True)
recognizer.fit(data["embeddings"], labels)

# write the actual face recognition model to disk
f = open("output/recognizer.pickle", "wb")
f.write(pickle.dumps(recognizer))
f.close()

# write the label encoder to disk
f = open("output/le.pickle", "wb")
f.write(pickle.dumps(le))
```

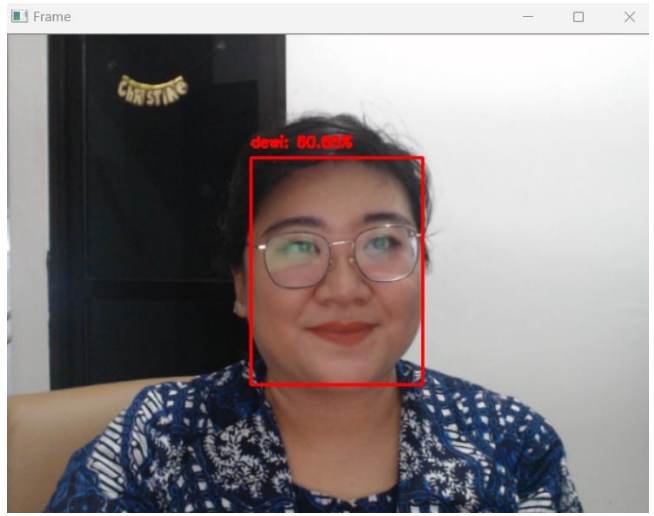
OUTPUT

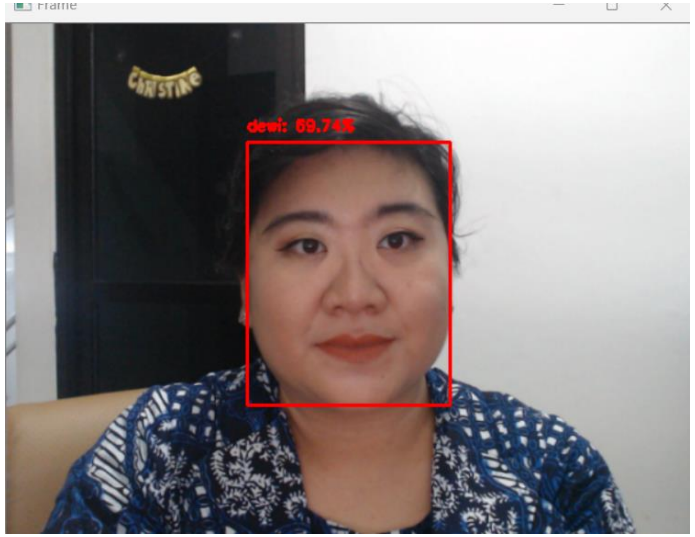
python recognize_image.py -i image2.jpg

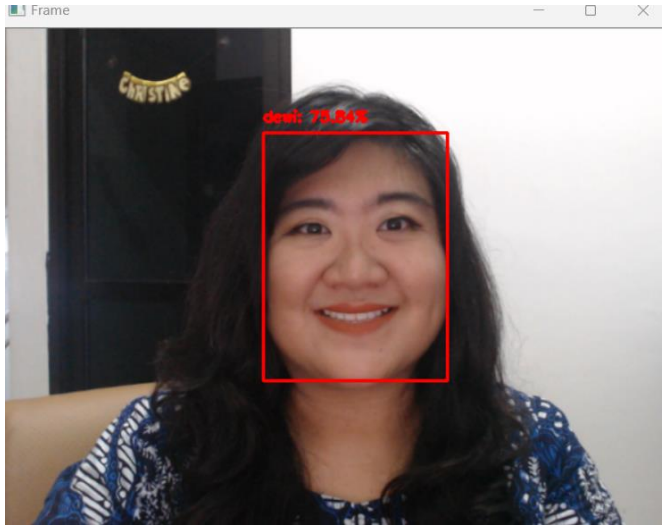


OUTPUT

python recognize_video.py







REFERENCE

1. https://github.com/italojs/facial-landmarks-recognition
2. https://github.com/aakashjhawar/face-recognition-using-deep-learning
3. https://github.com/aakashjhawar/face-detection
4. https://pyimagesearch.com/2017/04/03/facial-landmarks-dlib-opencv-python/
5. https://towardsdatascience.com/facial-mapping-landmarks-with-dlib-python-160abcf7d672
6. https://cmusatyalab.github.io/openface/

HOMEWORK

- Implement this code with your group member's face.
- Next week need to run the code and show it in the class.

☺ THANK YOU ☺

Christine Dewi, PhD
E-mail :
christine.dewi13@gmail.com
christine.dewi@uksw.edu,

Department of Information Management,
Chaoyang University of Technology
168, Jifong East Road, Wufong Dist., Taichung City 41349, Taiwan (R.O.C.)

Department of Information Technology
Satya Wacana Christian University
JL. Diponegoro 52 – 60, Salatiga 50711Central Java, Indonesia.



Website :
https://sites.google.com/view/christinedewi/home